

Programming for Finance

SQL

Aggregate functions

Department of Economics

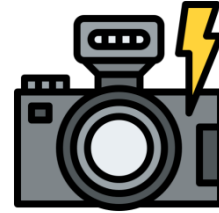
Ivan Bondarev



Email

- i.bondarev@inno.mgimo.ru

Signs



– Please take pictures of the information. Medium priority.

Source: https://www.flaticon.com/ru/free-icon/camera-flash_3460307



– Please write down the information. High priority.

Source: <https://openmoji.org/library/emoji-270D/>

TOPICS

1. Aggregate Functions
2. Grouping Data
3. Filtering Data
4. Reading Queries

Introduction to Aggregation

- Aggregation performs calculations on multiple rows
- Returns a **single value summarizing** the data
- Without aggregation you only see individual lines
- Aggregation turns data into analyzed information
- The most commonly used SQL **aggregate functions** are:
 - COUNT(), SUM(), AVG(), MIN(), MAX()

COUNT Function

- Counts the number of rows
- **COUNT()** counts all rows including NULLs
- **COUNT(*column*)** counts non-NULL values in that column
- *Use case:* Count number of transactions per day
- *Example:* **SELECT COUNT(column) FROM table**

SUM Function

- Calculates the total sum of a numeric column
- Ignores NULL values
- Helpful for totals (Total Sales, Total Expenses)
- ***Example:* SELECT SUM(amount) FROM sales**
- Ensure column is numeric type (int, float) to avoid errors

AVG Function

- Calculates the arithmetic mean of a numeric column
- Ignores NULL values
- ***Use case:*** Average transaction amount, Average salary
- ***Example:*** `SELECT AVG(price) FROM products`

MIN and MAX Functions

- **MIN()** returns the smallest value
- **MAX()** returns the largest value
- ***Use case:*** Highest single transaction, Lowest stock price
- ***Example:*** **SELECT MAX(amount), MIN(amount) FROM transactions**
- Works on dates too (earliest and latest date)

Aggregates and NULL Values

- Most aggregate functions ignore NULL
- **COUNT(*)** is the exception (counts rows regardless of NULL)
- **SUM(NULL)** results in NULL not zero
- Use **COALESCE** to handle NULLs in math
- *Example:* **SUM(COALESCE(amount, 0))**

Common Error: Mixing Aggregates

- Trying to sum a sum
- ***Example:*** `SUM(SUM(amount))`
- Usually not allowed directly
- Requires subquery or window function
- Keep aggregation levels clear



https://www.shutterstock.com/search/common-mistakes?image_type=vector

Aggregation without GROUP BY

- You can run an aggregate on the whole table
- ***Example:*** `SELECT SUM(revenue)`
`FROM sales`
- This returns one single row

1

GROUP BY Syntax

- **GROUP BY** groups rows that have the same values into summary rows
- Required when mixing aggregate functions with specific columns
- ***Example:*** Total sales per Region
- ***Syntax:*** **SELECT** region, **SUM**(sales) **FROM** data **GROUP BY** region

>1

GROUP BY Syntax

- Must appear after **WHERE** clause
- Must appear before **ORDER BY** clause
- Columns in **GROUP BY** must match non-aggregated columns in **SELECT**
- *Example:* **SELECT** dept, boss **SUM**(salary) **FROM** staff **GROUP BY** dept, boss
- Multiple columns allowed separated by commas

GROUP BY Syntax

- Every column in **SELECT** must be in **GROUP BY** or an aggregate
- You cannot select a raw column without grouping it
- ***Example Error:*** **SELECT** name, **SUM**(amount) **FROM** table
- ***Correct:*** **SELECT** name, **SUM**(amount) **FROM** table **GROUP BY** name
- Violating this causes a syntax error in most databases
- ***Fix:*** Add the column to GROUP BY
- ***Fix:*** Or wrap the column in an aggregate function

Grouping by Multiple Columns

- You can group by more than one dimension
- ***Example:*** Sales by Region and by Product
- ***Syntax:*** **GROUP BY** region, product
- Creates a hierarchy of aggregation

Filtering Before Aggregation (WHERE)

- **WHERE** clause filters individual rows before they are grouped
- Reduces the dataset before calculation happens
- ***Example:*** **WHERE** amount > 1000
- Cannot use aggregate functions in **WHERE** clause
- ***Example Error:*** **WHERE SUM**(amount) > 1000

Filtering After Aggregation (HAVING)

- **HAVING** clause filters groups after aggregation
- Used to filter on calculated values
- *Example:* **HAVING SUM(amount) > 1000**
- Must be used with **GROUP BY**
- Allows filtering on totals and averages

WHERE vs HAVING Comparison

- **WHERE** filters rows, **HAVING** filters groups
- **WHERE** happens before **GROUP BY**, **HAVING** after
- **WHERE** uses raw column values, **HAVING** uses aggregates
- *Example:* **WHERE** date > 2023 **GROUP BY** date **HAVING** **SUM**(sales) > 100
- Using the wrong one causes syntax errors

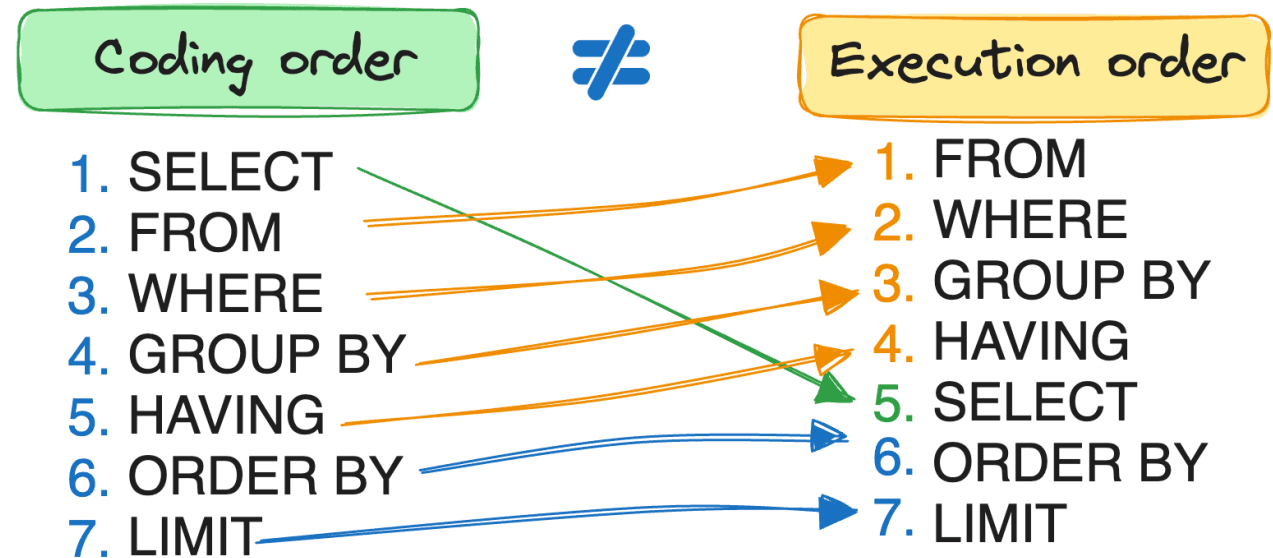
Aliases in SELECT

- Use **AS** to name columns or tables
- **Example:** **SUM**(amount) **AS** total_amount
- Improves readability of results
- Essential for financial reports requiring specific headers
- Aliases can be used in **ORDER BY**

Order of Execution in SQL

- Understanding order helps debug errors

- FROM and JOINS
- WHERE
- GROUP BY
- HAVING
- SELECT
- ORDER BY
- LIMIT

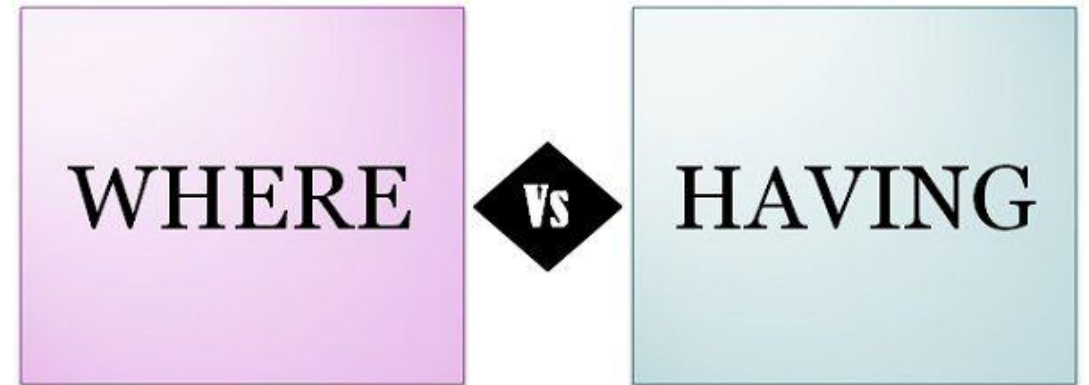


https://www.kdnuggets.com/wp-content/uploads/ferrer_essential_guide_sql_execution_order_3.png

- You cannot use column aliases in WHERE clause

Why Execution Order Matters

- You cannot filter on an *aggregate* in **WHERE**
- You must use **HAVING** for *aggregate* filters
- You cannot use **SELECT** alias in **WHERE**
 - Because **SELECT** happens after **WHERE**
- Example Error: **SELECT SUM(amount) AS total WHERE total > 100**
- Correct: **SELECT SUM(amount) AS total HAVING SUM(amount) > 100**
- Remember: **WHERE** for raw data, **HAVING** for summaries



<https://techdifferences.com/wp-content/uploads/2017/12/intro-image.jpg>

Summary of Key Commands

- **SELECT:** Choose columns
- **FROM:** Choose tables
- **WHERE:** Filter rows
- **GROUP BY:** Aggregate rows
- **HAVING:** Filter aggregates
- **JOIN:** Combine tables
- **ORDER BY:** Sort results



<https://www.freepik.com/free-photos-vectors/important-png>

Complete Query Example

- SELECT department, COUNT(*) as emp_count, AVG(salary) as avg_sal
- FROM employees
- WHERE status = 'Active'
- GROUP BY department
- HAVING AVG(salary) > 50000
- ORDER BY avg_sal DESC

Reading Complex Queries

- Start with **FROM** (where data comes from)
- Then check **WHERE** (what is filtered out)
- Then check **GROUP BY** (how it is summarized)
- Then check **SELECT** (what is shown)
- This helps understand existing code

Summary of Aggregation

- **COUNT, SUM, AVG, MIN, MAX** are core tools
- They convert rows into summary values
- NULL handling is important to understand



Summary of GROUP BY

- Groups data for aggregation
- Must match non-aggregated **SELECT** columns



- `SELECT COUNT (Passenger) AS n_of_people, place FROM Pass_in_trip
WHERE place LIKE '3%' GROUP BY place HAVING COUNT (Passenger)
>= 3`